

Paradigmi di Programmazione - A.A. 2025-26

Esame Scritto del 16/1/2026

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione usando la strategia **CALL-BY-NAME** fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita:

$$(\lambda x.x)((\lambda x.\lambda y.xy)y)$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & \underline{(\lambda x.x)((\lambda x.\lambda y.xy)y)} \\ & \text{nella **call-by-name** si passa l'argomento senza valutarlo} \\ \rightarrow & (\lambda x.\lambda y.xy)y \\ & \text{dato che la sostituzione } \{x = y\} \text{ andrebbe applicata a } (\lambda y.xy), \text{ ma } y \in FV(y) \\ \equiv_{\alpha} & \underline{(\lambda x.\lambda k.xk)y} \\ \rightarrow & \lambda k.yk \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y = match y with
  | (0,_,a) -> a=0
  | (_,_,a) -> if x then a=0 else x;;
```

SOLUZIONE:

Struttura del tipo:

$X \rightarrow Y \rightarrow \text{RIS}$

Uso per convenzione X, Y, Z come variabili di tipo per i parametri x, y, z , RIS come variabile di tipo del risultato, e $A, B, C, \dots$ come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
Y = int * B * A      (da pattern matching)
A = int               (da a=0)
RIS = bool            (da a=0)
X = bool              (da guardia dell'if)
```

Ne consegue:

```
X = bool
Y = int * B * bool
RIS = bool
```

Tipo inferito:

```
bool -> (int * B * int) -> bool
```

in sintassi OCaml:

```
bool -> (int * 'a * int) -> bool
```

Esercizio 3 [Punti 7]

Assumendo il seguente tipo di dato che descrive alberi binari di interi:

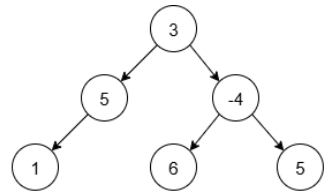
```
type btree =
| Void
| Node of int * btree * btree
```

si definisca, usando i costrutti di programmazione funzionale di OCAML:

- una funzione `tree_map` con tipo `tree_map: (int->int) -> btree -> btree` tale che `tree_map f bt` restituisca un albero ottenuto applicando la funzione `f` a ciascun valore intero contenuto nell'albero `bt`, preservandone la struttura.
- una funzione `tree_map_sum` con tipo `tree_map_sum: (int->int) -> btree -> btree * int` tale che `tree_map_sum f bt` restituisca una coppia composta da:
 - l'albero ottenuto applicando la funzione `f` a ciascun valore intero contenuto in `bt` usando la funzione `tree_map` del punto precedente;
 - la somma di tutti i valori presenti nell'albero ottenuto dopo l'applicazione di `tree_map`.

Esempio Si consideri il seguente albero binario (a destra in una rappresentazione visuale):

```
let bt = Node (3, Node (5,  
    Node (1, Void, Void), Void  
),  
    Node (-4,  
    Node (6, Void, Void), Node (5, Void, Void)  
))
```

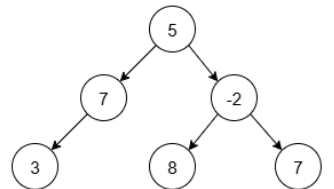


e la funzione

```
let inc2 x = x + 2
```

L'espressione `tree_map inc2 bt` restituisce l'albero:

```
Node (5, Node (7,  
    Node (3, Void, Void), Void  
),  
    Node (-2,  
    Node (8, Void, Void), Node (7, Void, Void)  
))
```

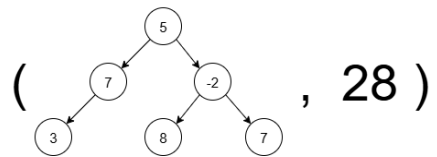


Nel caso della funzione `tree_map_sum`, l'espressione

```
tree_map_sum inc2 bt
```

restituisce una coppia composta dall'albero ottenuto applicando `inc2` a tutti i valori di `bt` e dalla somma dei valori dell'albero trasformato. Nel caso in esame, il risultato è:

```
( (* ALBERO OTTENUTO APPLICANDO tree_map *)  
  Node (5,Node (7,  
    Node (3, Void, Void),Void  
  ),  
    Node (-2,  
    Node (8, Void, Void), Node (7, Void, Void)  
  ) ),  
  28 )      (* SOMMA DEI VALORI *)
```



SOLUZIONE:

```
TREE_MAP

let rec tree_map f t =
  match t with
  | Void -> Void
  | Node (n, left, right) ->
      Node (f n,
            tree_map f left,
            tree_map f right)

TREE_MAP_SUM

let tree_map_sum f t =
  let rec sum tree =
    match tree with
    | Void -> 0
    | Node (n, left, right) -> n + sum left + sum right
  in let t1 = tree_map f t in
    (t1, sum t1);;

(* soluzione alternativa *)

let rec tree_map_sum f t =
  match t with
  | Void -> (Void, 0)
  | Node (n, left, right) ->
      let n' = f n in
      let (left', sumL) = tree_map_sum f left in
      let (right', sumR) = tree_map_sum f right in
      (Node (n', left', right'), n' + sumL + sumR)
```

Esercizio 4 [Punti 15]

Estendere il linguaggio MiniCaml visto a lezione con un nuovo tipo di dato che descrive **dizionari ad indice intero**, intesi come collezioni di associazioni (**chiave**, **valore**), dove ogni chiave è un numero intero e il valore è un valore del linguaggio.

Si assuma inoltre che i dizionari debbano essere **omogenei**: tutti i valori contenuti in un dizionario devono avere lo stesso tipo. Ad esempio, un dizionario può contenere solo valori interi $\{(2, 200), (1, 100)\}$ oppure solo valori booleani $\{(1, \text{true}), (3, \text{false})\}$, ma non una combinazione dei due. Si supponga che i dizionari possano contenere solo valori interi oppure solo valori booleani.

Si mostri come estendere la sintassi e modificare l'interprete del linguaggio MiniCaml con i seguenti costrutti:

- **DictInit** (*key_exp*, *val_exp*), che crea un dizionario contenente una sola associazione tra la chiave ottenuta dalla valutazione di *key_exp* e il valore ottenuto dalla valutazione di *val_exp*. Il costrutto deve sollevare un'eccezione se la valutazione di *key_exp* non produce un intero, né un booleano;
- **DictAdd** (*dict_exp*, *key_exp*, *val_exp*), che rappresenta il dizionario ottenuto aggiungendo una nuova associazione (*k*, *v*), dove *k* e *v* sono i risultati delle valutazioni di *key_exp* e *val_exp*, al dizionario *d*, ottenuto dalla valutazione di *dict_exp*. Il costrutto deve sollevare un'eccezione in ciascuno dei seguenti casi:
 1. la valutazione di *dict_exp* non produce un dizionario;
 2. la valutazione di *key_exp* non produce un intero;
 3. il tipo di *v* è diverso dal tipo dei valori già presenti.

- `DictGet (dict_exp, key_exp)`, che restituisce il valore associato alla chiave `k` (ottenuta dalla valutazione di `key_exp`) nel dizionario `d`, ottenuto dalla valutazione di `dict_exp`, sollevando un'eccezione in ciascuno dei seguenti casi:
 1. la valutazione di `dict_exp` non produce un dizionario;
 2. la valutazione di `key_exp` non produce un intero;
 3. la chiave non è presente nel dizionario.

Se il dizionario contiene più associazioni con la stessa chiave `k`, l'operazione `DictGet` restituisce il valore dell'associazione inserita più recentemente, ossia quella aggiunta per ultima al dizionario.

Nota: L'omogeneità del dizionario viene garantita dinamicamente confrontando, durante la valutazione, il tipo del nuovo valore con quello del primo valore già presente.

Esempio (in una sintassi concreta simile a OCaml):

```
let d1 = DictInit (1, 100);;          (* d1 = {(1, 100)} *)
let dk = DictInit (true, 100);;      (* ERRORE: chiave non intera *)
let d2 = DictAdd (d1, 2, 200);;      (* d2 = {(2, 200), (1, 100)} *)
let d3 = DictAdd (d2, 2, 300);;      (* d3 = {(2, 300), (2, 200), (1, 100)} *)
let d4 = DictAdd (d3, 3+2, 150-30);; (* d3 = {(5, 120), (2, 300), (2, 200), (1, 100)} *)
let d5 = DictAdd (d4, 3, true);;     (* ERRORE: valore non omogeneo *)
let d6 = DictAdd (d4, true, 400);;   (* ERRORE: chiave non intera *)
let v1 = DictGet (d4, 2);;           (* v1 = 300 *)
let v2 = DictGet (d4, 10);;          (* ERRORE: chiave non presente *)
```

SOLUZIONE:

Una possibile soluzione:

```
type exp =
  ...
  | DictInit of exp * exp
  | DictAdd of exp * exp * exp
  | DictGet of exp * exp;;

type evT =
  ...
  | DictVal of (int * evT) list;;

let rec eval e s =
  match e with
  ...

  | DictInit (ke, ve) ->
    (match eval ke s with
     | Int k ->
       let v = eval ve s in
       (match v with
        | Int _ -> DictVal [(k, v)]
        | Bool _ -> DictVal [(k, v)]
        | _ -> failwith "ERRORE: valore non booleano o intero")
     | _ -> failwith "ERRORE: chiave non intera"
    )

  | DictAdd (de, ke, ve) ->
    let d = eval de s in
    let k = eval ke s in
    let v = eval ve s in
    match v with
    | Int _ | Bool _ -> ...
    | _ -> failwith "ERRORE: valore non booleano o intero" in
    (match (d, k) with
     | (DictVal pairs, Int kv) ->
       (match pairs with
        | (_, firstV)::_ ->
          (match (v, firstV) with
           | (Int _, Int _) ->
             DictVal ((kv, v) :: pairs)
           | (Bool _, Bool _) ->
             DictVal ((kv, v) :: pairs)
           | _ ->
             failwith "ERRORE: valore non omogeneo"
          )
        | _ ->
          failwith "ERRORE: chiave non intera"
       )
     | _ ->
       failwith "ERRORE: non un dizionario"
    )

  | DictGet (de, ke) ->
    (match (eval de s, eval ke s) with
     | (DictVal pairs, Int kv) ->
       (match List.filter (fun (key, _) -> key = kv) pairs with
        | (_, v)::_ -> v
        | [] -> failwith "ERRORE: chiave non presente"
       )
     | (DictVal _, _) ->
       failwith "ERRORE: chiave non intera"
     | _ ->
       failwith "ERRORE: non un dizionario"
    )
)
```

SOLUZIONE:

VERSIONE ALTERNATIVA di DictGet

```
| DictGet (de, ke) ->
  let rec dict_get pairs k =
    match pairs with
    | [] -> failwith "ERRORE: chiave non presente"
    | (key, v) :: rest ->
      if key = k then v
      else dict_get rest k
  in
  (match (eval de s, eval ke s) with
  | (DictVal pairs, Int kv) ->
    dict_get pairs kv
  | (DictVal _, _) ->
    failwith "ERRORE: chiave non intera"
  | _ ->
    failwith "ERRORE: non un dizionario"
  )
)
```

SOLUZIONE:

VERSIONE ALTERNATIVA CON CONTROLLO DI TIPI

```
let getType (x : evT) : tname =
  match x with
  | Int _ -> TInt
  | Bool _ -> TBool
  | _ -> TUnBound

let typecheck ((x, y) : (tname * evT)) =
  match x with
  | TInt ->
    (match y with
     | Int _ -> true
     | _ -> false)
  | TBool ->
    (match y with
     | Bool _ -> true
     | _ -> false)
  | _ -> false

| DictInit (ke, ve) ->
  (match eval ke s with
   | Int k ->
     let v = eval ve s in
     if typecheck(v,TInt) || typecheck(v,TBool) then
       DictVal [(k, v)]
     else failwith "ERRORE: valore non booleano o intero")
  | _ -> failwith "ERRORE: chiave non intera"
  )

| DictAdd (de, ke, ve) ->
  let d = eval de s in
  let k = eval ke s in
  let v = eval ve s in
  (match (d, k) with
   | (DictVal pairs, Int kv) ->
     (match pairs with
      | [] ->
        DictVal [(kv, v)]
      | (_, firstV)::_ ->
        if getType v = getType firstV then
          DictVal ((kv, v) :: pairs)
        else
          failwith "ERRORE: valore non omogeneo"
     )
   | (DictVal _, _) ->
     failwith "ERRORE: chiave non intera"
   | _ ->
     failwith "ERRORE: non un dizionario"
  )
  )
```